



From Large Language Models to Reasoning and Agents

Miroslaw Staron^{ID} and Silvia Abrahão^{ID}

SOFTWARE ENGINEERING IS evolving rapidly thanks to generative AI and large language models (LLMs). Despite their well-known limitations, LLMs are here to stay, as they already provide unprecedented improvements in productivity and quality. At the same time, the underlying technology continues to evolve quickly: models get bigger, cloud provisioning becomes faster, and generation capabilities improve through explicit reasoning mechanisms. On top of that, we observe the rise of LLM-based agents, which are systems that combine language models with access to external tools to automate increasingly complex tasks and further accelerate software engineering workflows.

In this column, we analyze the ongoing transition from standalone LLMs to reasoning models and LLM-based agents. We summarize recent research that exemplifies this trend, drawing on papers from the 40th IEEE/ACM Conference on Automated Software Engineering (ASE), held in Seoul, Korea, in 2025. We welcome your feedback and suggestions on the topics covered. Additionally, if you experiment with

or adopt any of the practices featured here, please share your experiences with us, and the authors of the respective papers.

LLMs are in the Mainstream, but Reasoning and Agents are Gaining Traction

Automation is the central theme of the ASE conference. Papers in the main research track tackle fundamental software engineering problems and apply a variety of automation techniques to improve development speed, efficiency, and software quality. Figure 1 presents the frequency of automation mechanisms applied in the studies presented in the main research track. The largest category consists of studies that use LLMs either as standalone solutions or as core components of larger systems. These works address tasks, such as vulnerability detection, program repair, and software design. The second-largest category is fuzzing, which is primarily applied to software testing.

Particularly noteworthy, however, is the growing category of agents. In these studies, LLMs are augmented with access to tools and execution environments to direct them. Rather than merely suggesting fixes or

explanations, agent-based systems actively fix programs, run test cases, or attempt to reproduce defects. This shift moves LLMs from passive assistants toward more autonomous collaborators that can directly support software engineers in practice.

In this column, we dive deeper into studies that employ agentic AI and reasoning models to automate software engineering tasks. These approaches are especially interesting because they provide *design-level* intelligence and adaptive behavior that go beyond what can be achieved with standalone LLMs.

Let's Talk Gaming, Seriously

Computer games are an extremely interesting application domain because their goals differ significantly from those of many other software systems. They are meant for entertainment and often have relatively shorter lifespans than software used in domains, such as automotive systems, aviation, or critical infrastructure. At the same time, they are extremely demanding in terms of performance and stability. We hate it when our games crash just as we are about to strike the killing blow to the last dragon. Fixing these crashes requires developers to

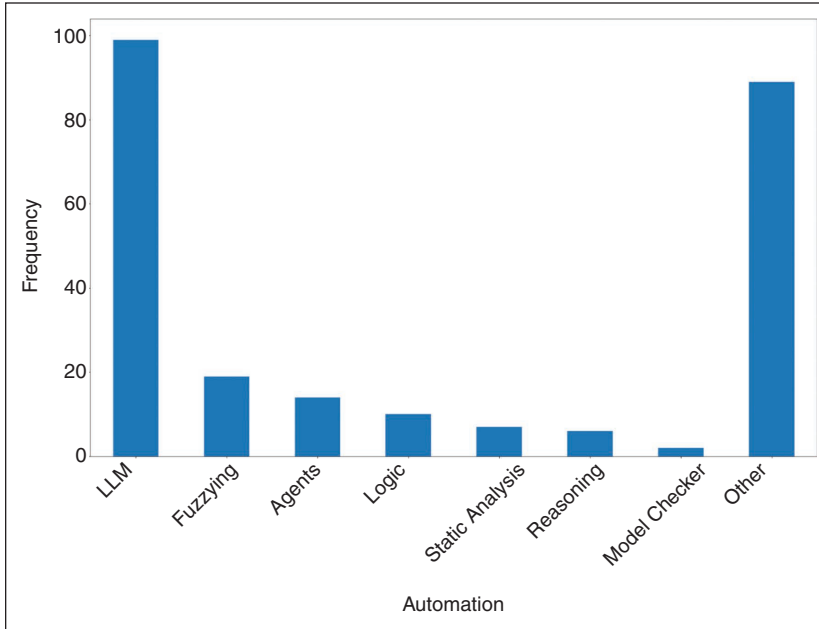


FIGURE 1. Distribution of automation mechanisms applied in papers at the 40th IEEE/ACM International Conference on ASE.

reproduce them first, a manual and difficult task. In the paper “*Agents in the Sandbox: End to-End Crash Bug Reproduction for Minecraft*,” Eray Yapağcı, Yavuz Alp Sencer Öztürk, and Eray Tüzün present BugCraft, a framework that uses AI agents to reproduce crash bugs in Minecraft. BugCraft operates in two steps. First, it analyzes user bug reports together with information from the Minecraft Wiki to outline a sequence of steps that are likely to reproduce the issue. Then, in the second step, the framework combines vision models with LLMs to play the game and to reproduce the bug. The second step is where agentic AI becomes essential, due to the inherent complexity and unpredictability of playing computer games. On BugCraft-Bench, a curated data set of Minecraft crash bug reports introduced in this study, BugCraft agent generated correct plans 66% of the time and achieved a 35%

end-to-end reproduction rate. The main obstacles to reproduction were limitations in the agent's ability to navigate complex 3D environments and minor logical errors in the generated plans. Although this technology has been applied to Minecraft, it can be used in a wide range of software systems that involve complex, interactive behavior. In such contexts, agentic AI can play a crucial role in automating the reproduction of complex bugs. The paper is available at <https://bugcraft2025.github.io/>.

Experts for Anomalies

Logfiles are prevalent artifacts of software engineering. Every professionally developed software should have them, as they help understand the software's runtime environment and identify problems that arise over its lifecycle. However, logfiles can evolve over time, just like the software's environment. The longer

the software lives, the more the log-file formats evolve. To keep up with the evolution, we need log parsers and analyzers that are flexible and can keep pace with the evolving logfiles. In the paper “*LogMoE: Lightweight Expert Mixture for Cross-System Log Anomaly Detection*,” Jiaxing Qi, Zhongzhi Luan, Shaohan Huang, Carol Fung, Yuchen Wang, Aibin Wang, Hongyu Zhang, Hailong Yang, Depei Qian, present such a log analyzer. LogMoE uses a mixture-of-experts architecture to reduce the need for accurate parsing of the log file before analysis. LogMoE was evaluated on eight data sets against six baselines. For the most challenging evaluation scenario, where LogMoE was trained on one system's logs and evaluated on another, it achieved F1 scores between 0.71 and 0.82. For single-system predictions trained and predicting different parts of the same log file, the system achieved F1 scores of 0.93 - 0.99. This technique is very promising, as it allows saving effort in parsing unknown log files and therefore makes the maintenance of large software systems easier. The paper is available at <https://tinyurl.com/mm6m89sk>.

Expert, Expert in the Room, Tell Me What This Code Does

Software engineers increasingly handle multiple programming languages in software projects. However, as the complexity of software systems grows, so does the complexity of the code handled in those different languages. On top of that, when we use generative AI for support, we need better models and better copilots. In the paper “*Mixture-of-Experts Low-Rank Adaptation for Multilingual Code Summarization*,”

Tianchen Yu, Li Yuan, Hailing Huang, Jiexin Wang, and Yi Cai present a new way of summarizing code by a special way of training a mixture-of-experts model. In

Ruby as well as on high-resource languages like Java and Python. The worst results were achieved with PHP and Go, though they were not much worse than the best

such as DeepSeek or StarCoder. The paper is available at <https://tinyurl.com/5xfwdn4m>.

What LLM Agents Do When They Fix Your Code

LLM-based agents are increasingly used to automate debugging and issue resolution, yet their internal decision-making remains opaque, making it difficult for practitioners to trust, diagnose, and improve them. In the paper “*Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories*,” Islem Bouzenia and Michael Pradel analyze how such agents actually work by studying their thought-action-result trajectories, which are the sequences of reasoning steps, tool invocations, and feedback they produce while repairing code. The authors conduct a

These studies suggest that the next generation of AI tooling will not merely recommend fixes, but execute tasks, adapt to context, and explain their decisions.

the experimental evaluations, the authors trained only 1.65% of all model parameters and achieved very good results on corner cases and low-resource languages like

baseline models. The main advantage of this technology, however, is the ability to achieve such results by training only 1.65% of the parameters in large models,

Table 1. Insights from selected papers presented at ASE 2025.

Research focus	Key insights	Takeaways
Game-based crash reproduction	Agents use bug reports, domain knowledge, vision models, and LLMs to play Minecraft. Bugcraft framework generated correct plans in 66% of cases and achieved a 35% end-to-end reproduction rate. Errors stem mainly from agent navigation limits or flawed plans.	Agentic testing can automate reproduction of complex interactive failures beyond games, including highly interactive systems.
Cross-system log anomaly detection	Mixture-of-experts models reduce reliance on rigid parsing and generalize across systems, achieving F1 scores of up to 0.82 cross-system and 0.99 within-system.	Learning-based log analysis can cut maintenance effort where formats evolve frequently.
Parameter-efficient multilingual code summarization	Training only 1.65% of parameters yields strong results across languages, including low-resource ones, with small drops for PHP and Go.	Lightweight fine-tuning enables customized copilots without retraining large models.
Understanding debugging agents' behavior	Trajectory analysis exposes success patterns (explore-fix-test) and failure modes (loops, skipped validation), revealing cost-accuracy trade-offs across agent designs.	Instrument agents and enforce testing cycles to improve reliability and control compute cost.
Context-aware code reuse in IDEs	Retrieval-augmented infilling, combined with static analysis, rewrites pasted code with >90% precision/recall and low latency.	IDE agents can eliminate much manual snippet adaptation when paired with compiler feedback.
Cognitive observability for agents	Post-hoc reasoning reconstruction highlights faulty assumptions and supports debugging, with small but consistent gains in downstream tasks.	Observability layers increase trust and diagnosability of LLM agents in production.

large-scale empirical study of three state-of-the-art agents (RepairAgent, AutoCodeRover, and OpenHands) using 120 real bug-fixing and issue-resolution runs. They unify heterogeneous logs into a common representation and combine quantitative metrics (iterations, token usage, action patterns) with qualitative coding of semantic relationships between thoughts, actions, and outcomes. They also mine recurring action sequences to uncover successful workflows and failure “antipatterns.” The study reveals that effective trajectories balance exploration, explanation, fix generation, and testing, whereas failed runs often exhibit repetitive loops, premature fixes without validation, or weak use of feedback. Even rare mismatches between an agent’s stated intent and its executed action strongly correlate with failure or excessive cost. Different architectures show distinctive tradeoffs: test-driven agents are thorough but expensive, while streamlined retrieve-and-fix workflows are cheaper but risk skipping validation. The authors release both data and annotation tools to support further research. The paper is available at <https://tinyurl.com/a7rbuwh9>.

Teaching IDEs to Rewrite Reused Code

Copy-paste-modify remains a dominant development practice, yet reused snippets often fail to compile because local variables, methods, or expressions do not match the surrounding context. This “code wiring” problem (replacing unresolved elements with suitable context-aware alternatives) still relies largely on manual effort or brittle heuristics. In the paper “*Wired for Reuse: Automating Context-Aware Code Adaptation in IDEs via LLM-Based*

Agent,” Taiming Wang, Yanjie Jiang, Chunhao Dong, Yuxia Zhang, and Hui Liu introduce WIRL, an LLM-based agent designed to automate this adaptation inside modern integrated development environments. WIRL frames code wiring as a retrieval-augmented infilling task rather than direct rewriting. It combines a language model with a customized tool kit for compiler-driven error detection, abstract syntax tree-based context collection, type filtering, and lexical-similarity ranking, orchestrated by an agent pilot that follows a lightweight state machine to avoid unproductive exploration. Implemented as an IntelliJ IDEA plugin, the approach iteratively gathers just enough context before asking the LLM to complete unresolved elements. Evaluated on a curated data set of real Stack Overflow-to-GitHub reuse cases, WIRL achieves 91.7% exact-match precision and 90.0% recall, outperforming raw LLMs, advanced code-editing systems, and IntelliJ’s own renaming support by large margins.

It also maintains practical latency and negligible per-use cost, while ablation studies show that its tooling and state-guided control are critical to these gains. The paper is available at <https://tinyurl.com/33z8ww78>.

Seeing Inside AI Agents

LLM-powered software agents increasingly automate development tasks, yet their reasoning remains opaque, making failures hard to diagnose and limiting trust. In the paper “*Watson: A Cognitive Observability Framework for the Reasoning of LLM-Powered Agents*,” Benjamin Rombaut, Sogol Masoumzadeh, Kirill Vasilevski, Dayi Lin, and Ahmed E. Hassan introduce the concept of *cognitive observability*, arguing that traditional logs and performance metrics are insufficient for understanding why such “Agentware” systems behave as they do. The authors present Watson, a framework that reconstructs an agent’s reasoning after execution without modifying its behavior. Watson deploys a surrogate agent configured to mirror the original model

ABOUT THE AUTHORS



MIROSLAW STARON is a full professor in the Interaction Design and Software Engineering division in the Computer Science and Engineering Department shared by Chalmers University of Technology and the University of Gothenburg, 412 96 Göteborg, Sweden. Contact him at <https://www.staron.nu> or mirosław.staron@cse.gu.se.



SILVIA ABRAHÃO is a full professor at the Department of Computer Systems and Computation, Universitat Politècnica de València, Valencia 46022, Spain. Contact her at <https://sabrahao.wixsite.com/dsic-upv> or sabrahao@dsic.upv.es.

and decoding parameters, generates candidate reasoning paths using fill-in-the-middle or repeated chain-of-thought prompting, and filters them through prompt-attribution analysis and an LLM-based judge so that retained explanations align with the prompt components that actually influenced the agent's output. The framework was evaluated on multiple-choice tasks from the massive multitask language understanding benchmark and on autonomous debugging agents (i.e., AutoCodeRover and OpenHands) over Software Engineering Benchmark-bench-lite. Case studies show that Watson's reconstructed traces help developers identify incorrect assumptions, guide targeted hints, and modestly improve

automated debugging pipelines, even when aggregate accuracy gains are not statistically significant. The paper is available at <https://tinyurl.com/45r5nyp6>.

This column examined how software engineering research at ASE 2025 is moving beyond standalone LLMs toward reasoning models and agentic systems that actively test, debug, adapt, and monitor software. The six papers synthesized in [Table 1](#) highlight a common shift toward design-level intelligence: agents that reproduce crashes in complex environments, models that generalize across evolving logs, parameter-efficient

copilots for multilingual code, trajectory analysis to understand automated debugging, IDE-integrated code adaptation, and observability layers that expose hidden reasoning. Together, these studies suggest that the next generation of AI tooling will not merely recommend fixes, but execute tasks, adapt to context, and explain their decisions—capabilities that are essential for safe adoption in professional software development. For practitioners, the message is clear: successful deployment of agentic AI will depend on combining LLMs with static analysis, testing loops, lightweight fine-tuning, and strong observability to ensure reliability, cost control, and trust in production systems. 



RECOGNIZE EXCELLENCE

COMPUTER SCIENCE & ENGINEERING AWARDS

IEEE Computer Society honors groundbreaking achievements in research and contributions to the global computing community.

AWARDS INCLUDE:

- Technical Achievements
- Teaching & Education
- Student Distinction & Scholarships
- Exemplary Service



To learn more, visit
computer.org/awards



**IEEE
COMPUTER
SOCIETY**

Digital Object Identifier 10.1109/MS.2026.3685785